

openGauss数据库SQL语法

难度（最高三星）：★★★ 建议学时（2-4学时）：4学时

实验说明

学习地图

本项目实验案例集是实现一个学生成绩管理系统，共分为9个实验任务，按照难度等级分为初级、中级、高级，如图1所示。

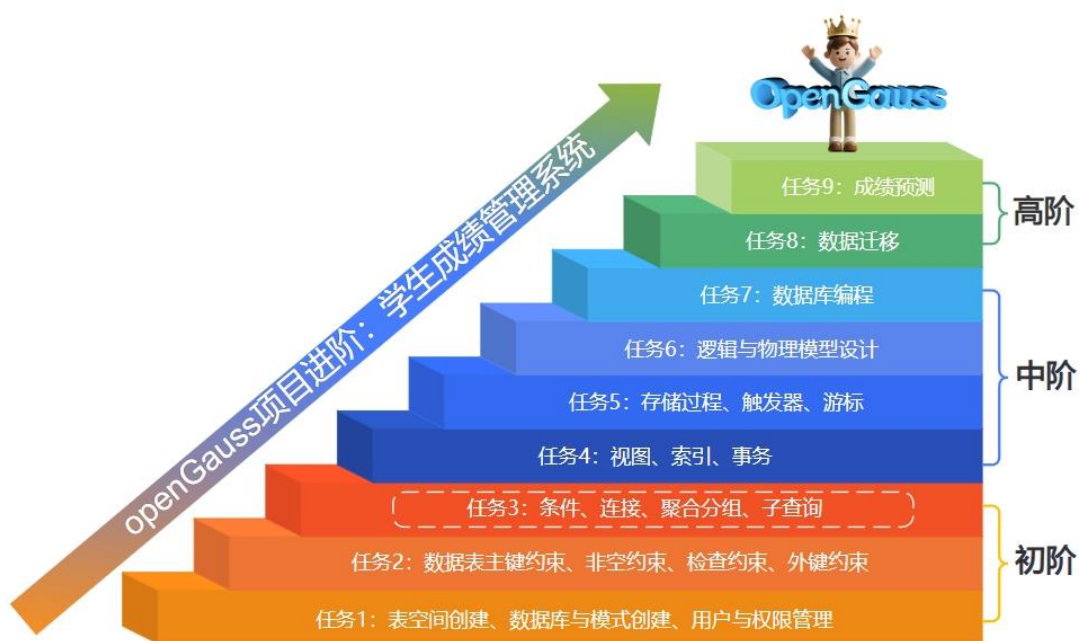


图1 学生成绩管理系统实验

本次实验主要完成其中的任务3，主要是使用SQL语句对学生成绩管理系统的数据表进行增删改查。

实验环境准备

(1) openGauss的安装与部署：该部分请参照实验任务1【[openGauss表空间、模](#)

[式、权限管理](#)】进行安装与部署。

(2) 数据库创建：请参照实验任务1【[openGauss表空间、模式、权限管理](#)】，或[直接下载相关SQL语句](#)进行数据库表空间创建、数据库模式创建、用户权限管理。

(3) 学生成绩管理系统的数据表以及相关约束：请参照实验任务2【[openGauss数据表创建与数据表约束](#)】，或[直接下载相关SQL语句](#)并进行运行生成相关的表。

实验任务

任务描述

本次实验共计六个子任务，主要是完成对学生成绩管理系统中的数据表使用SQL语句进行增删改查。整个实验会有详细的需求说明、实施步骤和关键代码。

学习目标

完成本任务的学习后，你应当能：

1. 学会使用 SQL 语句进行数据插入；
2. 学会使用 SQL 语句进行简单查询；
3. 学会使用 SQL 语句进行聚合与分组。

任务准备

前置知识

本实验案例，需提前学习openGauss的基本概念、作用及基础操作。可通过如下途径进行学习：

- 数字教材: [《数据库原理与应用: 基于openGauss》](#)
- 技术文档: [openGauss白皮书](#)

任务实施

任务1: 使用SQL语句为四张表添加数据

本实验的具体操作可参见视频。

训练要点: 使用SQL语句插入数据

需求说明: 根据数据库设计, 为学生成绩管理系统的表添加数据。

实现思路及关键代码:

--向Grade表插入数据

```
INSERT INTO Grade(grade_name) VALUES('大一年级');  
INSERT INTO Grade(grade_name) VALUES('大二年级');  
INSERT INTO Grade(grade_name) VALUES('大三年级');  
INSERT INTO Grade(grade_name) VALUES('大四年级');
```

--向Subject表插入数据

```
INSERT INTO Subject(subject_name,class_hour,grade_id) VALUES('数据库原理及应用',48,1);  
INSERT INTO Subject(subject_name,class_hour,grade_id) VALUES('数据库高级特性',32,3);  
INSERT INTO Subject(subject_name,class_hour,grade_id) VALUES('Python入门',36,1);  
INSERT INTO Subject(subject_name,class_hour,grade_id) VALUES('Java编程基础',36,1);  
INSERT INTO Subject(subject_name,class_hour,grade_id) VALUES('Java数据库实战',48,3);  
INSERT INTO Subject(subject_name,class_hour,grade_id) VALUES('C语言基础',36,1);  
INSERT INTO Subject(subject_name,class_hour,grade_id) VALUES('C语言进阶',48,2);  
INSERT INTO Subject(subject_name,class_hour,grade_id) VALUES('Python网络爬虫',36,2);  
INSERT INTO Subject(subject_name,class_hour,grade_id) VALUES('C语言实战进阶',56,3);  
INSERT INTO Subject(subject_name,class_hour,grade_id) VALUES('Git基础与实操',56,4);  
INSERT INTO Subject(subject_name,class_hour,grade_id) VALUES('计算机网络',96,2);
```

```
INSERT INTO Subject(subject_name,class_hour,grade_id) VALUES('计算机组成原理',96,2);
INSERT INTO Subject(subject_name,class_hour,grade_id) VALUES('操作系统',96,3);
INSERT INTO Subject(subject_name,class_hour,grade_id) VALUES('系统分析与设计',48,3);
INSERT INTO Subject(subject_name,class_hour,grade_id) VALUES('软件工程',56,4);
```

--向Student表插入数据

```
INSERT INTO Student VALUES('10000','Alice','女',1,13900000001,'天津市河西区','1987-09-08
00:00:00','Alice@123.com',1100000000000000001);
INSERT INTO Student VALUES('10001','Bob','男',1,13900000002,'地址不详','1993-03-04
00:00:00','Bob@123.com',1100000000000000002);
INSERT INTO Student VALUES('10002','Charlie','女',1,13900000003,'河南洛阳','1992-08-08
00:00:00','Charlie@123.com',1100000000000000003);
INSERT INTO Student VALUES('10011','David','男',1,13900000004,'地址不详','1984-12-31
00:00:00','David@123.com',1100000000000000004);
INSERT INTO Student VALUES('10012','Emma','女',1,13900000005,'地址不详','1991-08-30
00:00:00','Emma@123.com',1100000000000000005);
INSERT INTO Student VALUES('20011','Frank','男',2,13900000006,'北京市海淀区','1994-01-01
00:00:00','Frank@123.com',1100000000000000006);
INSERT INTO Student VALUES('20012','Grace','女',2,13900000007,'北京市东城区','1990-06-05
00:00:00','Grace@123.com',1100000000000000007);
INSERT INTO Student VALUES('30015','Hannah','女',3,13900000008,'河北省石家庄市','1996-
10-01 00:00:00','Hannah@123.com',1100000000000000008);
INSERT INTO Student VALUES('30021','Isabella','女',3,13900000009,'上海市卢湾区','1995-09-
29 00:00:00','Isabella@123.com',1100000000000000009);
INSERT INTO Student VALUES('40023','Jacob','男',4,13900000010,'广州市天河区','1997-08-30
00:00:00','Jacob@123.com',1100000000000000010);
```

--向Result表插入数据

```
INSERT INTO Result VALUES('10001',1,76,'2024-02-15 00:00:00');
INSERT INTO Result VALUES('10000',3,60,'2024-02-17 00:00:00');
INSERT INTO Result VALUES('10001',3,46,'2024-02-15 00:00:00');
INSERT INTO Result VALUES('10001',3,76,'2024-02-17 00:00:00');
```

```
INSERT INTO Result VALUES('10002',6,83,'2024-02-17 00:00:00');
INSERT INTO Result VALUES('10011',3,71,'2024-02-16 00:00:00');
INSERT INTO Result VALUES('10011',4,95.5,'2024-02-17 00:00:00');
INSERT INTO Result VALUES('10012',4,76,'2024-02-15 00:00:00');
INSERT INTO Result VALUES('20011',7,60,'2024-02-15 00:00:00');
INSERT INTO Result VALUES('20012',8,91,'2024-02-15 00:00:00');
INSERT INTO Result VALUES('20012',11,61,'2024-02-15 00:00:00');
INSERT INTO Result VALUES('30015',2,60,'2024-02-15 00:00:00');
INSERT INTO Result VALUES('30015',5,65,'2024-02-15 00:00:00');
INSERT INTO Result VALUES('30021',2,23,'2024-02-15 00:00:00');
INSERT INTO Result VALUES('30021',5,74,'2024-02-15 00:00:00');
INSERT INTO Result VALUES('40023',10,23,'2024-02-15 00:00:00');
INSERT INTO Result VALUES('40023',15,39,'2024-02-15 00:00:00');
```

任务2： 使用SQL语句在学生成绩管理系统中做基础查询练习

本实验的具体操作可参见视频。

训练要点：使用SQL语句做基础查询练习。

需求说明：根据数据库设计，在学生成绩管理系统中做基础查询练习，要求如下：

- (1) 查询所有学生的姓名和性别。
- (2) 查询每门科目的名称和课时数。

实现思路及关键代码：

- (1) 查询所有学生的姓名和性别，结果如图2所示。

```
SELECT student_name, sex
FROM student;
```

	student_name	sex
1	Alice	女
2	Bob	男
3	Charlie	女
4	David	男
5	Emma	女
6	Frank	男
7	Grace	女
8	Hannah	女
9	Isabella	女
10	Jacob	男

图2 查询所有学生的姓名和性别

(2) 查询每门科目的名称和课时数，如图3所示。

```
SELECT subject_name, class_hour
FROM subject;
```

	subject_name	class_hour
1	数据库原理及应用	48
2	数据库高级特性	32
3	Python入门	36
4	Java编程基础	36
5	Java数据库实战	48
6	C语言基础	36
7	C语言进阶	48
8	Python网络爬虫	36
9	C语言实战进阶	56
10	Git基础与实操	56
11	计算机网络	96
12	计算机组成原理	96
13	操作系统	96
14	系统分析与设计	48
15	软件工程	56

图3 查询每门科目的名称和课时数

任务3：使用SQL语句在学生成绩管理系统中做条件查询练习

本实验的具体操作可参见视频。

训练要点：使用SQL语句做条件查询练习。

需求说明：根据数据库设计，在学生成绩管理系统中做条件查询练习，要求如下：

- (1) 查询以“D”开头和以“a”结尾的学生信息。
- (2) 查询课时数大于等于50的科目名称和课时数。

实现思路及关键代码：

- (1) 查询以“D”开头和以“a”结尾的学生信息，如图4所示。

```
SELECT *
FROM student
WHERE student_name LIKE 'D%'
OR student_name LIKE '%a';
```

	student_no	student_name	sex	grade_id	phone	address	born_date	email	identity_card
1	10011	David	男	1	13900000004	地址不详	1984-12-31 00:00:00	David@123.com	1100000000000000004
2	10012	Emma	女	1	13900000005	地址不详	1991-08-30 00:00:00	Emma@123.com	1100000000000000005
3	30021	Isabella	女	3	13900000009	上海市卢湾区	1995-09-29 00:00:00	Isabella@123.com	1100000000000000009

图4 查询以“D”开头和以“a”结尾的学生信息

- (2) 查询课时数大于等于50的科目名称和课时数，如图5所示。

```
SELECT subject_name, class_hour
FROM subject
WHERE class_hour >= 50;
```

	subject_name	class_hour
1	C语言实战进阶	56
2	Git基础与实操	56
3	计算机网络	96
4	计算机组成原理	96
5	操作系统	96
6	软件工程	56

图5 查询课时数大于等于50的科目名称和课时数

任务4：使用SQL语句在学生成绩管理系统中做连接查询练习

本实验的具体操作可参见视频。

训练要点：使用SQL语句做连接查询练习

需求说明：根据数据库设计，在学生成绩管理系统中做连接查询练习，要求如下：

- (1) 查询每个学生的姓名、所在年级和年级名称。
- (2) 查询每个学生的姓名、所选科目和对应的成绩。
- (3) 查询每门科目的名称、所属年级和该科目的平均成绩（如果有成绩记录）。

实现思路及关键代码：

- (1) 查询每个学生的姓名、所在年级和年级名称，如图6所示。

```
SELECT s.student_name, g.grade_name
FROM Student s
JOIN Grade g ON s.grade_id = g.grade_id;
```

	student_name	grade_name
1	Alice	大一年级
2	Bob	大一年级
3	Charlie	大一年级
4	David	大一年级
5	Emma	大一年级
6	Frank	大二年级
7	Grace	大二年级
8	Hannah	大三年级
9	Isabella	大三年级
10	Jacob	大四年级

图6 查询每个学生的姓名、所在年级名称

- (2) 查询每个学生的姓名、所选科目和对应的成绩，如图7所示。

```
SELECT s.student_name, sub.subject_name, r.student_result
```



```
FROM student s
JOIN result r ON s.student_no = r.student_no
JOIN subject sub ON r.subject_No = sub.subject_no;
```

	student_no	subject_name	student_result
1	Alice	Python入门	60
2	Bob	数据库原理及应用	76
3	Bob	Python入门	46
4	Bob	Python入门	76
5	Charlie	C语言基础	83
6	David	Python入门	71
7	David	Java编程基础	96
8	Emma	Java编程基础	76
9	Frank	C语言进阶	60
10	Grace	Python网络爬虫	91
11	Grace	计算机网络	61
12	Hannah	数据库高级特性	60
13	Hannah	Java数据库实战	65
14	Isabella	数据库高级特性	23
15	Isabella	Java数据库实战	74
16	Jacob	Git基础与实操	23
17	Jacob	软件工程	39

图7 查询每个学生的姓名、所选科目和对应的成绩

(3) 查询每门科目的名称、所属年级和该科目的平均成绩（如果有成绩记录），平均成绩只保留小数点后两位，如图8所示。

```
SELECT sub.subject_name, g.grade_name, ROUND(AVG(r.Student_Result), 2) AS
average_score
FROM Subject sub
JOIN result r ON sub.subject_no = r.subject_no
JOIN student s ON r.student_no = s.student_no
JOIN grade g ON sub.grade_id = g.grade_id
GROUP BY sub.subject_name, g.grade_name;
```

	subject_name	grade_name	average_sc...
1	数据库原理及应用	大一年级	76.00
2	Git基础与实操	大四年级	23.00
3	数据库高级特性	大三年级	41.50
4	Java数据库实战	大三年级	69.50
5	Java编程基础	大一年级	86.00
6	C语言进阶	大二年级	60.00
7	Python入门	大一年级	63.25
8	计算机网络	大二年级	61.00
9	Python网络爬虫	大二年级	91.00
10	C语言基础	大一年级	83.00
11	软件工程	大四年级	39.00

图8 查询每门科目的名称、所属年级和该科目的平均成绩

任务5：使用SQL语句在学生成绩管理系统中做聚合函数和分组练习。

本实验的具体操作可参见视频。

训练要点：使用SQL语句做聚合函数和分组练习。

需求说明：根据数据库设计，在学生成绩管理系统中做聚合函数和分组练习，要求如下：

- (1) 查询每个年级的学生数量。
- (2) 查询每个年级的平均成绩。
- (3) 查询每门科目的最高分、最低分和平均分。

实现思路及关键代码：

- (1) 查询每个年级的学生数量，如图9所示。

```
SELECT g.grade_name, COUNT(*) AS student_count
FROM student s
JOIN grade g ON s.grade_id = g.grade_id
GROUP BY g.grade_name;
```

	grade_name	student_co...
1	大四年级	1
2	大二年级	2
3	大一年级	5
4	大三年级	2

图9 查询每个年级的学生数量

(2) 查询每个年级的平均成绩，如图10所示。

```
SELECT g.grade_name, AVG(r.Student_Result) AS average_score
FROM grade g
JOIN Subject sub ON g.grade_id = sub.Grade_Id
JOIN Result r ON sub.Subject_No = r.Subject_No
JOIN student s ON r.Student_No = s.student_no
GROUP BY g.grade_name;
```

	grade_name	average_score
1	大一年级	73.0000000000000000
2	大四年级	31.0000000000000000
3	大二年级	70.6666666666666667
4	大三年级	55.5000000000000000

图10 查询每个年级的平均成绩

(3) 查询每门科目的最高分、最低分和平均分，如图11所示。

```
SELECT sub.Subject_Name, MAX(r.Student_Result) AS max_score, MIN(r.Student_Result) AS
min_score, AVG(r.Student_Result) AS average_score
FROM Subject sub
JOIN Result r ON sub.Subject_No = r.Subject_No
JOIN student s ON r.Student_No = s.student_no
GROUP BY sub.Subject_Name;
```

	subject_name	max_score	min_score	average_score
1	C语言基础	83	83	83.0000000000000000
2	Git基础与实操	23	23	23.0000000000000000
3	Python网络爬虫	91	91	91.0000000000000000
4	数据库原理及应用	76	76	76.0000000000000000
5	软件工程	39	39	39.0000000000000000
6	数据库高级特性	60	23	41.5000000000000000
7	Java编程基础	96	76	86.0000000000000000
8	Java数据库实战	74	65	69.5000000000000000
9	Python入门	76	46	63.2500000000000000
10	计算机网络	61	61	61.0000000000000000
11	C语言进阶	60	60	60.0000000000000000

图11 查询每门科目的最高分、最低分和平均分

任务6：使用SQL语句在学生成绩管理系统中做子查询练习

本实验的具体操作可参见视频。

训练要点：使用SQL语句做子查询练习。

需求说明：根据数据库设计，在学生成绩管理系统中做子查询练习，要求如下：

- (1) 查询每个学生的总成绩和平均成绩，同时显示每个学生的排名（按总成绩降序排列）。
- (2) 查询每门科目的平均分，并显示高于平均分的学生人数。
- (3) 查询成绩最高的学生信息（包括姓名和总成绩）。

实现思路及关键代码：

- (1) 查询每个学生的总成绩和平均成绩，同时显示每个学生的排名（按总成绩降序排列），如图12所示。

```
SELECT s.Student_No, s.Student_Name,
       (SELECT SUM(r.Student_Result) FROM Result r WHERE r.Student_No = s.Student_No)
AS total_score,
```

```

        (SELECT AVG(r.Student_Result) FROM Result r WHERE r.Student_No = s.Student_No)
AS average_score,
        (SELECT COUNT(*) + 1 FROM (SELECT SUM(r2.Student_Result) AS total_score FROM
Result r2 GROUP BY r2.Student_No HAVING total_score > total_score) AS rank WHERE
rank.total_score > total_score) AS rank
FROM student s
ORDER BY total_score DESC;

```

	student_no	student_name	total_score	average_score	rank
1	10001	Bob	198	66.0000000000000000	1
2	10011	David	167	83.5000000000000000	1
3	20012	Grace	152	76.0000000000000000	1
4	30015	Hannah	125	62.5000000000000000	1
5	30021	Isabella	97	48.5000000000000000	1
6	10002	Charlie	83	83.0000000000000000	1
7	10012	Emma	76	76.0000000000000000	1
8	40023	Jacob	62	31.0000000000000000	1
9	20011	Frank	60	60.0000000000000000	1
10	10000	Alice	60	60.0000000000000000	1

图12 查询每个学生的总成绩和平均成绩，同时显示每个学生的排名

(2) 查询每门科目的平均分，并显示高于平均分的学生人数，如图13所示。

```

SELECT
    s.Subject_Name,
    (SELECT AVG(r.Student_Result)
    FROM Result r
    WHERE r.Subject_No = s.Subject_No) AS average_score,
    (SELECT COUNT(*)
    FROM Result r2
    WHERE r2.Subject_No = s.Subject_No
    AND r2.Student_Result > (SELECT AVG(r.Student_Result)
    FROM Result r
    WHERE r.Subject_No = s.Subject_No)) AS
above_average_count
FROM Subject s;

```

	subject_name	average_score	above_aver...
1	数据库原理及应用	76.0000000000000000	0
2	数据库高级特性	41.5000000000000000	1
3	Python入门	63.2500000000000000	2
4	Java编程基础	86.0000000000000000	1
5	Java数据库实战	69.5000000000000000	1
6	C语言基础	83.0000000000000000	0
7	C语言进阶	60.0000000000000000	0
8	Python网络爬虫	91.0000000000000000	0
9	C语言实战进阶	[NULL]	0
10	Git基础与实操	23.0000000000000000	0
11	计算机网络	61.0000000000000000	0
12	计算机组成原理	[NULL]	0
13	操作系统	[NULL]	0
14	系统分析与设计	[NULL]	0
15	软件工程	39.0000000000000000	0

图13 查询每门科目的平均分，并显示高于平均分的学生人数

(3) 查询成绩最高的学生信息（包括姓名和总成绩），如图14所示。

```
SELECT s.Student_Name,
       (SELECT SUM(r.Student_Result) FROM Result r WHERE r.Student_No = s.Student_No)
AS total_score
FROM student s
ORDER BY total_score DESC
LIMIT 1;
```

	student_na...	total_score
1	Bob	198

图14 查询成绩最高的学生信息

随堂测试

1. 查找岗位是工程师且薪水在6000以上的记录，逻辑表达式为（ ）

A. 岗位 = '工程师' OR 薪水 > 6000

B. 岗位 = 工程师 AND 薪水 > 6000

C. 岗位 = 工程师 OR 薪水 > 6000

D. 岗位 = '工程师' AND 薪水 > 6000

2. WHERE子句中, 表达式 "age between 20 and 30" 等同于 ()

A. age >= 20 and age <= 30

B. age >= 20 or age <=30

C. age > 20 and age < 30

D. age > 20 and age < 30

3. 从student表中查询学生姓名、年龄和成绩, 结果按照年龄降序排序, 年龄相同的按照成绩升序排序, 下面SQL语句正确的是 ()

A. select name, age, score from student order by age desc , score;

B. select name, age, score from student order by age , score asc;

C. select name, age, score from student order by 2 desc , 3 asc;

D. select name, age, score from student order by 1 desc , 2 ;

4. 使用SQL命令将STAFFS表中员工的年龄AGE字段的值增加5岁, 应该使用的命令是 ()

A. UPDATE SET AGE WITH AGE+5

B. UPDATE AGE WITH AGE+5

C. UPDATE STAFFS SET AGE = AGE+5

D. UPDATE STAFFS AGE WITH AGE+5

5. 下面四组SQL命令，全部属于数据操作语言的命令是（ ）

- A. CREATE, DROP, UPDATE
- B. INSERT, UPDATE, DELETE
- C. INSERT, DROP, ALTER
- D. UPDATE, DELETE, ALTER

6. 删除表student中班级(cid)为6的全部学生信息，下面SQL语句正确的是（ ）

- A. delete from student where cid = 6;
- B. delete * from student where cid = 6;
- C. delete from student on cid = 6;
- D. Delete * from student on cid = 6;